Network Mastery with AWS VPC Mini Project

Project Overview
Project Duration: 2 hours

In this session, we explore the core concepts of Amazon Web Services (AWS), focusing specifically on Virtual Private Clouds (VPCs). Our objective is to understand the fundamental components of VPC infrastructure, including subnets, gateways, and routing tables. Through practical demonstrations and interactive exercises, we navigate the AWS Management Console to deploy and manage these critical components effectively.

Before proceeding with setting up VPCs, ensure a solid understanding of cloud networking basics. If terms like "Cloud" are unfamiliar, review earlier materials to establish a strong foundation in cloud computing concepts.

Project Goals
- Understand the fundamentals of Virtual Private Cloud (VPC) and its components.
- Gain hands-on experience in setting up and configuring VPC, subnets, Internet Gateway, NAT Gateway, and VPC peering connections.
- Learn how to enable internet connectivity securely within a VPC.
- Implement outbound internet access through the NAT Gateway.
- Establish direct communication between VPCs using VPC peering.

Learning Outcomes
- Acquired knowledge about VPC and its essential components, such as subnets, gateways, and route tables.
- Developed skills in creating and configuring VPC resources using the AWS Management Console.
- Learned how to set up routing tables to enable internet connectivity and outbound internet access securely.
- Gained understanding of VPC peering and its significance in connecting multiple VPCs within the same or different regions.
- Enhanced understanding of network security principles and best practices for cloud environments.

Key Concepts

What is VPC, Subnets, Internet Gateway, and NAT Gateway?
Imagine building a virtual space for the company GatoGrowFast.com so that its computers can communicate securely. That's what a **Virtual Private Cloud (VPC)** is all about—a private room in the cloud for GatoGrowFast.com's use.

Example: Think of GatoGrowFast.com's office building with different departments like HR, Finance, and IT, each with its own area and access rules. In a VPC, GatoGrowFast.com creates different sections, called **subnets**, for different parts of the business.

To connect its office to the internet, GatoGrowFast.com uses a router to control data flow. In a VPC, this is achieved with an **Internet Gateway**, allowing secure communication with the internet.

A **NAT (Network Address Translation) Gateway** acts as a secret agent between GatoGrowFast.com's computers and the internet. When a computer in the virtual office communicates with the internet, the NAT Gateway translates the message and hides the sender's identity, similar to sending a letter without a return address. This keeps GatoGrowFast.com's computers safe and anonymous online.

Note: A **router** directs data packets between networks, acting like a traffic cop for the internet. Data is broken into packets, and the router uses **routing tables**—like maps of the internet—to determine the best path for these packets based on destination IP addresses.

What is an IP Address?
An **IP address** is like a phone number for a computer, a unique set of numbers that helps devices find and communicate with each other on a network, like the internet.

Types of IP Addresses:
- **Public IP Address**: Like a home address, it's unique and allows other computers on the internet to find your device. Assigned by an Internet Service Provider (ISP), it enables global communication. Public IPs can be **dynamic** (changing periodically) or **static** (constant, used for servers or consistent connectivity).
- **Private IP Address**: Like an internal extension number in an office, used for communication within a specific network (e.g., home Wi-Fi or office network). Assigned by a router or DHCP server, private IPs are not routable over the internet and can be reused across different private networks without conflict.

IP Address Versions:
- **IPv4**: The most common type, a 32-bit numeric address written in decimal format (e.g., 192.168.0.1). Each octet ranges from 0 to 255, divided into classes A, B, and C for host addressing.
- **IPv6**: Designed to replace IPv4 due to address exhaustion, a 128-bit hexadecimal address (e.g., 2001:0db8:85a3:0000:0000:8a2e:0370:7334), offering a vast address space.

What is CIDR?
CIDR (Classless Inter-Domain Routing) simplifies talking about groups of IP addresses. Instead of listing each address, CIDR uses a shorthand, like saying "all houses on Main Street" instead of naming each house.

Example: For the IP address 192.168.1.0, CIDR notation like 192.168.1.0/24 refers to all addresses from 192.168.1.0 to 192.168.1.255.

Calculating Available IP Addresses in a CIDR Block:
Use the formula: `2^(32 - CIDR notation) - 2` (subtracting 2 for the network and broadcast addresses).

Example: For 192.168.1.0/24:
- `2^(32 - 24) - 2 = 2^8 - 2 = 256 - 2 = 254` available IP addresses.

What is a Gateway?
Gateways are like doorways between networks, enabling data to travel between a local network and others, like the internet. They act as a traffic cop, directing data to its destination.

Example: In a city with neighborhoods, to visit a friend in Neighborhood B from Neighborhood A, you pass through a gateway connecting the two. Similarly, a network gateway connects your local network to the internet.

What is a Route Table?
A **route table** is like a map guiding data around a network. It lists destinations and paths (routes) to reach them. Devices consult the route table to determine where to send data packets.

Example: To send data to a website, a computer checks its route table to find the gateway to the internet. The router then forwards the data to the next stop, ensuring it reaches its destination.

Connection Between Gateway and Route Table
- **Gateways**: Devices (e.g., routers, firewalls) serving as entry/exit points between networks with different IP ranges. They receive packets and determine the next destination based on routing information.
- **Route Tables**: Maintained by networking devices, they list destination networks and the next hop (gateway) to reach them. Devices use route tables to find the best path for packets.
- **Connection**: When a device sends data outside its local network, it checks the route table to identify the gateway. The packet is forwarded to the gateway, which continues routing until the packet reaches its destination.

Difference Between Internet Gateway and NAT Gateway
- **Internet Gateway**: A door to the internet for a subnet, allowing resources (e.g., EC2 instances) to send and receive internet traffic. It enables two-way communication.
- **NAT Gateway**: A one-way street for subnet traffic, allowing resources to access the internet without allowing incoming internet traffic, enhancing security.

What is VPC Peering?
VPC peering connects two virtual offices (VPCs) in the cloud for direct communication, like neighboring offices sharing files without a middleman. By default, EC2 instances in different VPCs cannot communicate. VPC peering establishes a direct network connection between VPCs.

Why VPC Peering?
It enables different parts of a cloud network (e.g., development and marketing VPCs) to share data securely and efficiently.

Key Points:
- VPCs require peering, VPN, or AWS Direct Connect for connectivity.
- Subnets within the same VPC communicate by default via AWS-configured route tables.
- EC2 instances in the same or different subnets within a VPC communicate if security group rules and route tables allow.
- **VPC Peering Basics**:
  - Allows direct communication using private IP addresses.
  - Supports same or different regions and AWS accounts.
  - CIDR blocks must not overlap.
  - Requires proper Security Group and Network Access Control List (NACL)

```
configurations.
  - No transitive traffic (traffic cannot flow through a peered VPC to another
VPC).
  - Route tables must include routes for the peer VPC's CIDR block.
  - Limits exist on the number of peering connections and route entries.

### What is a VPC Endpoint?
A **VPC endpoint** is a secure, dedicated tunnel between a VPC and an AWS service
(e.g., S3), bypassing the public internet. It ensures private, safe access to
resources.

**Problem Context:** Backing up data from an EC2 instance to an S3 bucket
typically goes over the internet, risking sensitive data exposure. A VPC endpoint
creates a private connection, keeping data secure.

**Note:** An **EC2 instance** is a virtual server in AWS for running applications,
offering scalable computing power for hosting websites, software, or data
processing.

## Practical Steps

### Part 1: Setting Up a Virtual Private Cloud (VPC)
1. Navigate to the search bar, enter 'VPC', and click on the VPC service.
   ![Navigate to search bar and enter VPC]
(./img/1.Navigate_to_search_bar_and_enter_VPC_then_click_on_VPC_service_that_appea
rs.png)

2. On the VPC dashboard, click **Create VPC**.
   ![Click Create VPC](./img/2.On_the_VPC_dashboard_Click_Create_VPC.png)

3. Select **VPC only**, set the IPv4 CIDR block (e.g., 10.0.0.0/16), and click
**Create VPC**.
   ![Set VPC CIDR and create]
(./img/3.On_Create_VPC_page_Select_VPC_only_set_IPV4_CIDR_block_and_click_create_V
PC.png)

4. VPC successfully created.
   ![VPC created](./img/4.vpc_successfully_created.png)

5. **Note**: If a CIDR block size error occurs, ensure the block size is between
/16 and /28.
   ![CIDR block size note](./img/5.note-
in_case_of_CIDR_block_size_error_let_your_block_size_fall_within_the_specified_ran
ge.png)

### Part 2: Configuring Subnets within the VPC
1. From the created VPC, click **Subnets** to create a subnet.
   ![Click Subnets]
(./img/6.here_is_the_created_VPC_click_subnets_to_create_subnet.png)

2. On the subnet dashboard, click **Create subnet**.
   ![Create subnet](./img/7.on_the_subnet_dashboard_click_create_subnet.png)

3. Select the VPC created in Part 1.
```

```
   ![Select VPC](./img/8.select_the_Subnet_VPC(Created_in_the_first_step).png)

4. Enter subnet name (e.g., `my-public-subnet-1`), set availability zone, specify
IPv4 CIDR block (e.g., 10.0.1.0/24), and click **Add new subnet**.
   ![Create public subnet](./img/9.Enter_subnet_name(my-public-subnet-
1)_set_availability_zone_subnet_IPv4_CIDR_block_and_click_add_new_subnet.png)

5. Enter subnet name (e.g., `my-private-subnet-1`), set availability zone, specify
IPv4 CIDR block (e.g., 10.0.2.0/24), and click **Create subnet**.
   ![Create private subnet](./img/10.Enter_subnet_name(my-private-subnet-
1)_set_availability_zone_subnet_IPv4_CIDR_block_and_click_Create_subnet.png)

6. View the architecture.
   ![VPC architecture](./img/11.the_architecture.png)

7. Subnets `my-public-subnet-1` and `my-private-subnet-1` successfully created.
   ![Subnets created](./img/12.my-public-subnet-1_and_my-private-subnet-
1_successfully_created.png)

### Part 3: Creating Internet Gateway and Attaching it to VPC
1. Click **Internet Gateways** to connect the public subnet to the internet.
   ![Click Internet Gateways](./img/13.sincemy-public-subnet-1_and_my-private-
subnet-
1_successfully_created_now_click_on_internet_gateway_to_connect_public_subnet_to_t
he_internet.png)

2. Click **Create Internet Gateway**.
   ![Create Internet Gateway]
(./img/14.on_the_internet_gateway_dashboard_click_create_internet_gateway_button.p
ng)

3. Name the Internet Gateway and click **Create Internet Gateway**.
   ![Name Internet Gateway]
(./img/15.name_internet_gateway_and_click_create_internet_gateway.png)

4. Internet Gateway created, note it is detached.
   ![Internet Gateway detached]
(./img/16.internet_gateway_successfully_created_note_that_the_state_shows_detached
.png)

5. Click **Actions** and **Attach to VPC**.
   ![Attach to VPC](./img/17.click_Actions_and_then_Attach_to_VPC.png)

6. Select the VPC and click **Attach Internet Gateway**.
   ![Attach Internet Gateway]
(./img/18.select_vpc_and_click_attach_internet_gateway.png)

7. Internet Gateway successfully attached to VPC.
   ![Internet Gateway attached]
(./img/19.internet_gateway_successfully_attached_to_vpc.png)

### Part 4: Enabling Internet Connectivity with the Internet Gateway by Setting Up
Routing Tables
1. View the current architecture.
```

```
   ![Current architecture](./img/20.present_architechture.png)

2. Click **Route Tables**.
   ![Click Route Tables]
(./img/21.since_IGW_is_now_attached_to_vpc_click_on_route_tables.png)

3. Click **Create route table**.
   ![Create route table]
(./img/22.on_the_route_table_dashboard_click_create_route_table.png)

4. Name the route table, select the VPC, and click **Create route table**.
   ![Name route table]
(./img/23.name_route_table_select_vpc_and_click_create_route_table.png)

5. Click **Subnet associations**, then **Edit subnet associations**.
   ![Edit subnet associations]
(./img/24.route_table_created_now_click_on_subnet_associations_then_edit_subnet_as
sociation.png)

6. Select the public subnet and click **Save associations**.
   ![Save subnet associations]
(./img/25.select_public_subnet_and_click_save_associations.png)

7. Under **Routes** tab, click **Edit routes**.
   ![Edit routes]
(./img/26.subnet_association_successfully_saved_now_under_routes_tab_click_edit_ro
utes.png)

8. Click **Add route**.
   ![Add route](./img/27.click_add_route_to_add_new_route.png)

9. Set **Destination** to `0.0.0.0/0`, **Target** to the created Internet Gateway,
and save changes.
   ![Set route to Internet Gateway](./img/28.set_route_destination_to_0.0.0.0-
0_Target_to_the_created_IGW_and_save_changes.png)

10. Route table updated successfully.
    ![Route table updated](./img/29.route_table_updated_successfully.png)

### Part 5: Enabling Outbound Internet Access through NAT Gateway
1. View the current VPC architecture.
   ![VPC architecture](./img/30.present_vpc_or_network_architecture.png)

2. Click **NAT Gateways**.
   ![Click NAT Gateways]
(./img/31.now_that_route_tables_have_been_updated_click_on_NAT_Gateway.png)

3. Click **Create NAT Gateway**.
   ![Create NAT Gateway](./img/32.click_create_NAT_gateway.png)

4. Name the NAT Gateway, select the private subnet, set connectivity type to
**Private**, and click **Create NAT Gateway**.
   ![Configure NAT Gateway]
(./img/33.name_NATgw_select_private_subnet_and_set_connectivity_type_to_private.pn
```

```
g)

5. NAT Gateway created successfully.
   ![NAT Gateway created](./img/34.NAT_gw_created_successfully.png)

6. Select the NAT Gateway, locate the subnet ID in the **Details** tab, and click
it.
   ![Locate subnet ID]
(./img/35.select_the_created_NAT_gw_and_locate_subnet_id_link_at_details_section_a
nd_click_on_it.png)

7. Navigate to the **Route Table** section and click the route table ID.
   ![Click route table ID](./img/36.In_the_subnet
page_navigate_to_Route_Table_section_and_click_one_route_table_id(rtb-...).png)

8. Click **Routes**, then **Edit routes**.
   ![Edit routes]
(./img/37.on_route_table_page_click_on_Routes_tab_and_then_edit_routes.png)

9. Click **Add route**.
   ![Add route](./img/38.on_the_edit_route_page_click_add_route.png)

10. Set **Destination** to `0.0.0.0/0`, **Target** to **NAT Gateway**, select the
created NAT Gateway, and save changes.
    ![Set route to NAT Gateway](./img/39.select_0.0.0.0-
0_as_destination_NAT_Gateway_as_the_target_and_the_created_nat_gateway_as_the_NAT_
Gateway_then_Save_changes.png)

11. Route table updated successfully, click **Subnet associations**, then **Edit
subnet associations**.
    ![Edit subnet associations]
(./img/40.route_table_updated_successfully_now_click_subnet_association_and_edit_s
ubnet_association.png)

12. Select the private subnet and click **Save associations**.
    ![Save private subnet association]
(./img/41.click_on_the_private_subnet_and_then_save_associations_button.png)

13. Subnet successfully attached to the route table.
    ![Subnet attached]
(./img/42.the_subnet_has_been_successfully_attached_with_the_route_table..png)

14. View the current VPC architecture.
    ![Current VPC architecture](./img/43.present_vpc_architecture_or_status.png)

15. Create an Ubuntu EC2 instance named `Public-EC2`.
    ![Create Public-EC2]
(./img/44.create_an_Ubuntu_EC2_instance_and_name_it_Public-
EC2_and_scroll_down.png)

16. In network settings, click **Edit**.
    ![Edit network settings]
(./img/45.after_setting_the_key_pair_on_the_network_settings_click_on_edit.png)
```

17. Set VPC to 10.0.0.0/16, subnet to the public subnet, enable **Auto-assign public IP**, and launch instance.
    ![Launch Public-EC2](./img/46.set_its_VPC_to_the_created_vpc(10.0.0.0-16)_and_subnet_to_the_public_subnet_Enable_Auto-assign_public_IP_then_launch_instance.png)

18. Create an Ubuntu EC2 instance named `Private-EC2`.
    ![Create Private-EC2](./img/47.create_an_Ubuntu_EC2_instance_and_name_it_Private-EC2_and_scroll_down.png)

19. In network settings, click **Edit**.
    ![Edit network settings](./img/48.after_setting_the_key_pair_on_the_network_settings_click_on_edit.png)

20. Set VPC to 10.0.0.0/16, subnet to the private subnet, and launch instance.
    ![Launch Private-EC2](./img/49.set_its_VPC_to_the_created_vpc(10.0.0.0-16)_and_subnet_to_the_private_subnet_then_launch_instance.png)

21. Both EC2 instances created, click the `Public-EC2` ID.
    ![Click Public-EC2 ID](./img/50.both_EC2_now_created_click_on_the_public-EC2_id.png)

22. Copy the public IP for remote connection.
    ![Copy public IP](./img/51.copy_the_public_IP_for_a_remote_connection_over_the_internet.png)

23. Connect via SSH.
    ![SSH connection](./img/52.connected_via_ssh.png)

24. Test outbound traffic with `git clone`.
    ![Test outbound traffic](./img/53.git_clone_to_test_outbound_traffic.png)

25. Copy the public IP of `Private-EC2` for remote connection (note: connection should fail).
    ![Copy Private-EC2 IP](./img/54.copy_the_public_IP_of_the_PrivateEC2_as_well_for_a_remote_connection_over_the_internet.png)

26. Unable to connect to `Private-EC2` over the internet.
    ![Connection failure](./img/55.cant_connect_to_it_over_the_internet.png)

27. Verify NAT Gateway functionality by navigating to **NAT Gateways** and selecting the created NAT Gateway.
    ![Verify NAT Gateway](./img/56.to_verify_that_nat_gateway_is_doing_its_work_go_to_VPC_then_NAT_Gateways_and_click_on_the_created_nat_gateway.png)

28. Observe outbound traffic data in **Packet Out to Destination** and **Byte Out to Destination**.
    ![Outbound traffic data](./img/57.observe_that_Packet_Out_to_Destination_Byte_Out_to_Destination_generate_some_data.png)

29. Create another EC2 instance to test connectivity within the same subnet.
    ![Create another EC2]
(./img/58.create_another_EC2_to_test_connectivity_of_systems_in_the_same_subnet.png)

30. Set VPC to 10.0.0.0/16, subnet to the public subnet, and launch instance.
    ![Launch new EC2](./img/59.set_its_VPC_to_the_created_vpc(10.0.0.0-16)_and_subnet_to_the_public_subnet_then_launch_instance.png)

31. Two EC2 instances now in the public subnet.
    ![Two EC2s in public subnet]
(./img/60.there_are_now_two_ec2_in_the_public_subnet.png)

32. Click the new `Public-EC2`.
    ![Click new Public-EC2](./img/62.click_on_the_new_Public-EC2.png)

33. Click **Connect**.
    ![Click Connect](./img/63.on_the_dashboard_click_on_connect.png)

34. Select **EC2 Instance Connect** tab and click **Connect**.
    ![Connect via EC2 Instance Connect]
(./img/64.click_on_EC2_Instance_Connect_tab_and_then_Click_connect_button_to_connect.png)

35. Successfully connected via browser.
    ![Browser connection](./img/65.successfully_connected_via_the_browser.png)

36. Connect to the other EC2 in the public subnet using a terminal.
    ![Terminal connection]
(./img/66.connect_to_the_other_EC2_in_public_subnet_using_terminal_application.png)

37. Successfully connect via terminal.
    ![Terminal success](./img/67.successfully_connect_via_hyper_terminal.png)

38. Ping the second EC2 to ensure reachability.
    ![Ping EC2](./img/68.ping_EC2-2_using_EC2_to_ensure_reachability.png)

39. Move the key pair to EC2 using SCP to SSH into the second EC2.
    ![Move key pair]
(./img/69.move_the_keypair_to_EC2_using_SCP_in_order_to_use_it_to_SSH_into_EC2-2.png)

40. Confirm key pair copied and SSH into `Public-EC2-2` from `Public-EC2`.
    ![SSH into Public-EC2-2]
(./img/70.confirm_key_pair_file_copied_successfully_and_use_it_to_ssh_into_Public-EC2-2_from_Public-EC2.png)

41. Successfully SSH into `Public-EC2-2`.
    ![SSH success](./img/71.successfully_ssh_into_public-ec2-2_from_public-ec2.png)

42. Log out of both instances.
    ![Log out](./img/72.logged_out_of_both.png)

```
### Part 6: Establishing VPC Peering Connections
1. Create the requester VPC (192.168.0.0/16).
   ![Create requester VPC](./img/73.to_practice_VPC_Peering_create_the_requester-
VPC(192.168.0.0-16).png)

2. Create the accepter VPC (172.16.0.0/16).
   ![Create accepter VPC](./img/74.also_create_accepter_vpc(172.16.0.0-16).png)

3. Both VPCs created, navigate to **Peering Connections**.
   ![Navigate to Peering Connections]
(./img/75.both_vpc_successfully_created_navigate_to_peering_connection.png)

4. Click **Create Peering Connection**.
   ![Create Peering Connection](./img/76.Click_create_peering_connection.png)

5. Name the peering connection, select requester VPC, choose **My account**,
select accepter VPC, and click **Create Peering Connection**.
   ![Configure peering connection]
(./img/77.name_the_peering_connection_select_requester-
VPC_for_requester_and_choose_account_to_peer_with(My_account)_accepter-
VPC_for_accepter.png)

6. Scroll down and create the peering connection.
   ![Create peering](./img/78.scroll_down_and_create_peering_connection.png)

7. VPC peering connection created, click **Actions** and **Accept Request**.
   ![Accept peering request]
(./img/79.VPC_peering_connection_successfully_created_click_Actions_and_Accept_req
uest.png)

8. Click **Accept Request** in the pop-up.
   ![Confirm accept request](./img/80.In_the_pop-up_click_Accept_request.png)

9. VPC peering connection established, click **VPC**.
   ![VPC peering established]
(./img/81.vpc_peering_connection_has_been_established_click_on_vpc.png)

10. Click the accepter VPC's main route table ID.
    ![Click accepter route table ID]
(./img/82.click_on_accepter_vpc_main_route_table_id.png)

11. Click the route table ID.
    ![Click route table ID](./img/83.click_on_the_route_table_ID.png)

12. Click **Edit routes**.
    ![Edit routes](./img/84.click_on_Edit_routes.png)

13. Click **Add route**.
    ![Add route](./img/85.click_on_add_route.png)

14. Copy the IPv4 CIDR of the requester VPC (192.168.0.0/16).
    ![Copy requester CIDR]
(./img/86.note_that_you_need_to_copy_the_IPv4_CIDR_of_requester_VPC.png)
```

15. Add the requester VPC’s CIDR to the **Destination**, set **Target** to the peering connection, and save changes.
    ![Add route for accepter](./img/87.add_vpc_IPv4_CIDR_of_the_accepter_vpc_to the_destination_target_peering_connection_and_select_the_created_peering_then_save _changes.png)

16. Route table updated successfully, return to **VPC**.
    ![Route table updated] (./img/88.route_table_updatedsuccessfully_now_go_back_to_vpc.png)

17. Copy the IPv4 CIDR of the accepter VPC (172.16.0.0/16) and click the requester VPC’s main route table ID.
    ![Copy accepter CIDR] (./img/89.copy_the_IPv4_CIDR_of_accepter_VPC_and_click_on_the_main_route_table_of_ requester_vpc.png)

18. Click the route table ID.
    ![Click route table ID](./img/90.click_on_the_route_table_ID惆

System: The response was cut off due to exceeding the maximum length. Here's the continuation in markdown format:

```markdown
_table_ID.png)

19. Click **Edit route**.
    ![Edit route](./img/91.click_on_edit_route.png)

20. Add the accepter VPC’s CIDR to the **Destination**, set **Target** to the peering connection, and save changes.
    ![Add route for requester](./img/92.add_vpc_IPv4_CIDR_of_the_requester_vpc_to the_destination_target_peering_connection_and_select_the_created_peering_then_save _changes.png)

21. Peering connection successfully established.
    ![Peering successful](./img/93.peering_now_successful.png)

The connection has been successfully established. Now, resources in the accepter VPC can connect to resources in the requester VPC, and vice versa.

## Project Reflection
- Successfully completed the project tasks related to setting up VPC infrastructure and configuring network components.
- Gained practical experience in navigating the AWS Management Console and configuring VPC resources.
- Encountered challenges such as CIDR block size limitations and learned how to troubleshoot and resolve them effectively.
- Developed a deeper understanding of network architecture and cloud networking concepts through hands-on experimentation.
- Recognized the importance of network security measures, such as NAT Gateway and VPC endpoints, in ensuring secure communication within cloud environments.
```

```
Overall, the project offered valuable perspectives on the fundamentals of cloud
networking and practical experience in deploying VPC infrastructure on AWS.
```